Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😁

alkemy

Clase grabada

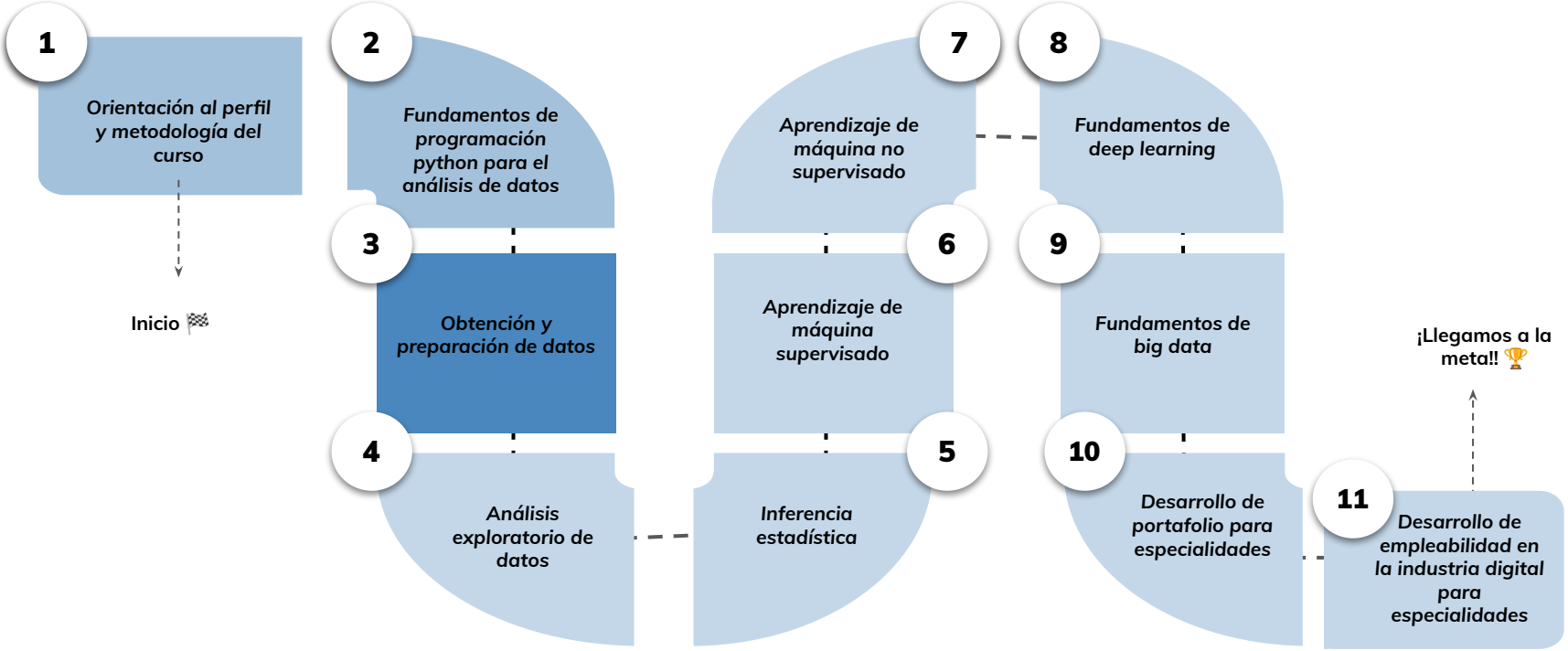
› Agrupamiento y pivoteo de datos - Parte II

Aprendizaje Esperado 6: Aplicar técnicas de unión, agrupamiento y pivoteo de datos utilizando la librería pandas para el reacomodo de la información para la resolución de un problema.

alkemy

Hoja de ruta

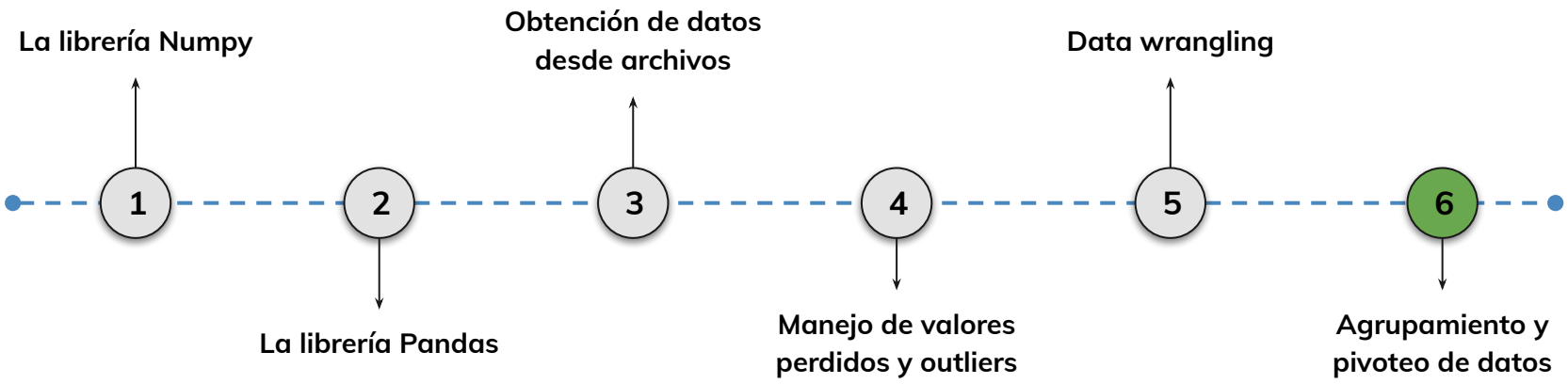
¿Cuáles skills conforman el programa? **Fundamentos de Ciencia de Datos**



alkemy

Roadmap de lecciones

*¿Cuáles **lecciones** estaremos estudiando en este módulo?*



alkemy

Learning Path

¿Cuáles temas trabajaremos hoy?

6.

Filtrado y automatización de resúmenes

Podrás realizar filtrado, ordenamiento y limpieza posterior al agrupamiento, generando reportes automáticos listos para ser exportados. Aplicarás condicionales, manejo de nulos y formateo de resultados.

- **Post-procesamiento de agrupaciones**
  - **Manejo de nulos**
  - **Filtros y ordenamientos**
- **Automatización y exportación**
  - **Reportes automáticos**
  - **Exportación de resultados**

alkemy

Objetivos de aprendizaje

¿Qué aprenderás?

- Utilizar valores nulos en tablas dinámicas.
- Formatear resultados y aplicar filtros avanzados.
- Preparar reportes automatizados con Pandas.

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos 📚:

→ *Agrupamiento por columnas.*

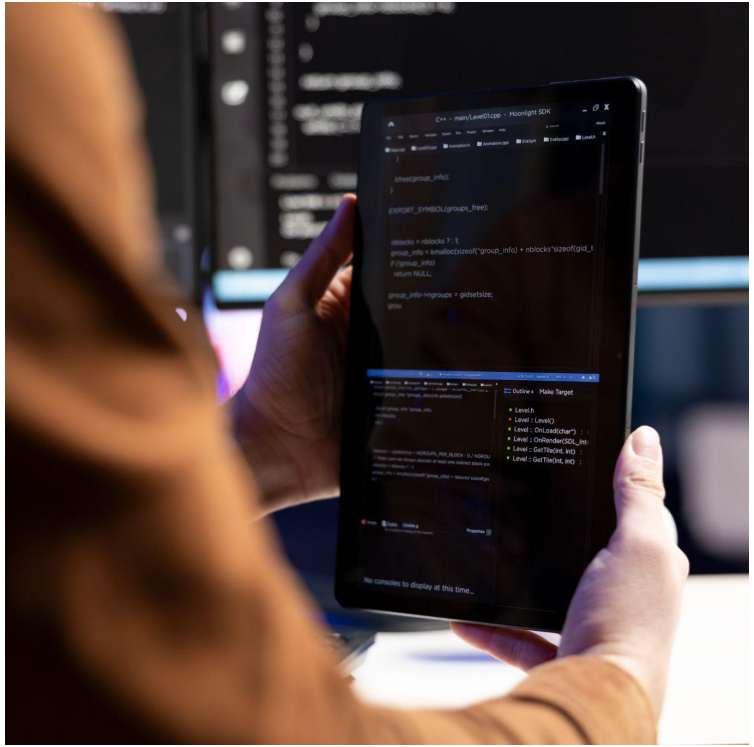
→ *Funciones de agregación y resumen.*

→ *Uso de .agg() y .pivot_table().*

Merge de dataframes

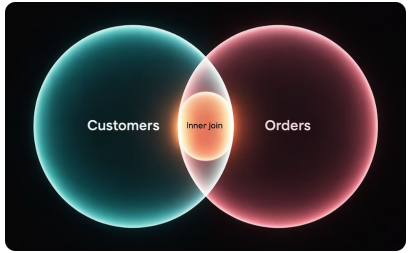
MERGE DE DATAFRAMES

El método merge() permite combinar dos DataFrames en base a una clave común, similar a una unión en bases de datos SQL. Es una técnica esencial cuando se trabaja con datos estructurados en múltiples tablas o fuentes de información.



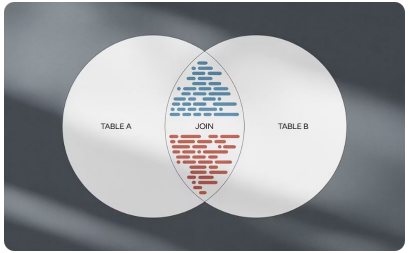
Tipos de combinaciones (merge())

Existen diferentes tipos de combinaciones que pueden realizarse con merge() en Pandas:



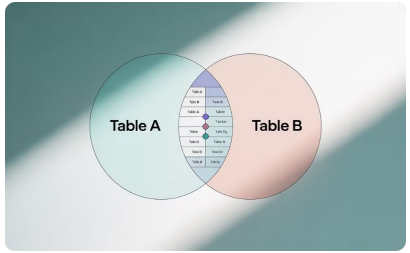
Inner join

Solo conserva los registros con coincidencias en ambas tablas.



Left join

Mantiene todos los registros de la tabla izquierda y añade coincidencias de la tabla derecha.



Right join

Conserva todos los registros de la tabla derecha y añade coincidencias de la tabla izquierda.

Outer join

Mantiene todos los registros de ambas tablas, rellenando con valores nulos donde no haya coincidencias.

Ejemplo de uso de merge()

En este ejemplo, se combinan dos DataFrames (clientes y ventas) utilizando la columna 'ID_Cliente' como clave común. El resultado es un nuevo DataFrame que contiene información de ambas tablas para los clientes que tienen registros de ventas.

```
# Crear dos DataFrames
df1 = pd.DataFrame({'ID': [1, 2, 3], 'Nombre': ['Ana', 'Luis', 'Carlos']})
df2 = pd.DataFrame({'ID': [1, 2, 4], 'Salario': [50000, 54000, 60000]})

# Realizar un merge por la columna 'ID'
df_merged = pd.merge(df1, df2, on='ID', how='inner')
print(df_merged)
```

Beneficios del uso de merge()

Integración de datos

Facilita la integración de datos provenientes de distintas fuentes, permitiendo análisis más completos.

Análisis completos

Permite realizar análisis más completos al combinar información complementaria de diferentes tablas.

Estructuración relacional

Ayuda a estructurar datos en modelos relacionales sin redundancia, optimizando el almacenamiento.

Manipulación eficiente

Optimiza la manipulación de grandes volúmenes de información en entornos analíticos complejos.

Parámetros importantes de merge()

on

Especifica la columna o columnas que se utilizarán como clave para la unión.

how

Define el tipo de unión: 'inner', 'left', 'right', 'outer'.

left_on y right_on

Permite especificar columnas con nombres diferentes en cada DataFrame para la unión.

suffixes

Define los sufijos para columnas con el mismo nombre en ambos DataFrames.

indicator

Añade una columna que indica la fuente de cada fila en el resultado.

validate

Verifica la integridad de la unión, asegurando relaciones 1:1, 1:m, m:1 o m:m.

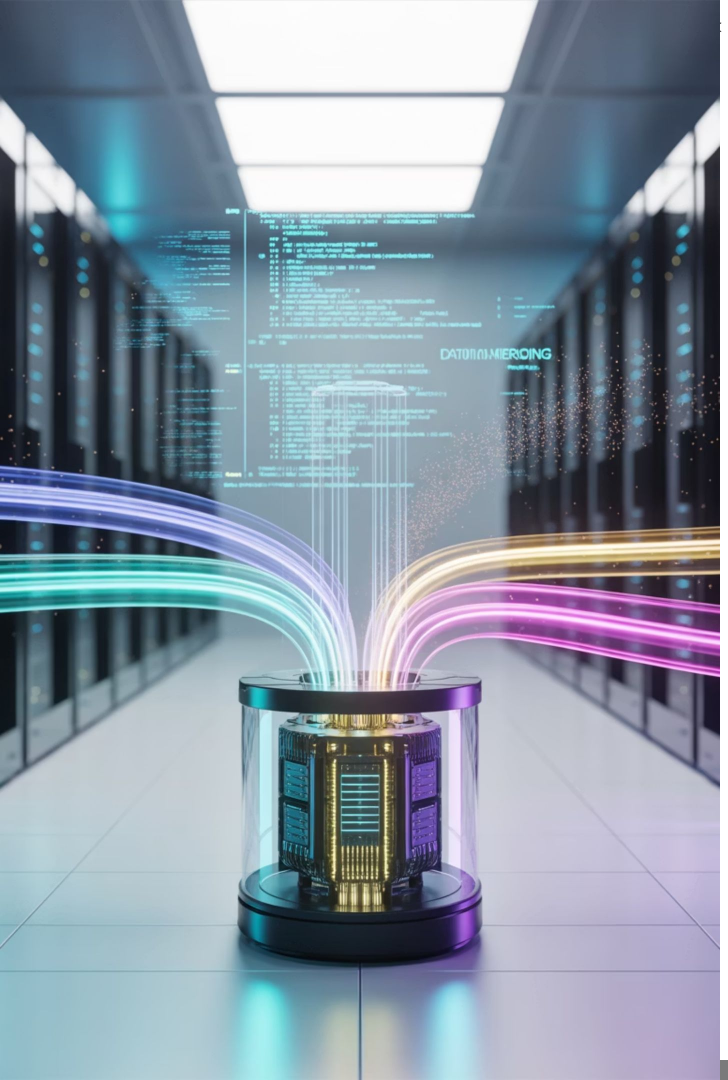
Ejemplo práctico: Análisis de ventas y clientes

Supongamos que tenemos dos DataFrames:

- **ventas**: contiene información sobre transacciones (ID_Venta, ID_Cliente, Producto, Monto, Fecha)
- **clientes**: contiene información sobre clientes (ID_Cliente, Nombre, Email, Región)

Para analizar el comportamiento de compra por región, podríamos:

```
# Combinar los DataFramesdf_combinado =
pd.merge( ventas,  clientes,
on='ID_Cliente',  how='left')# Agrupar por
región y calcular
estadísticasanalisis_region =
df_combinado.groupby('Región').agg({
'Monto': ['sum', 'mean', 'count'],
'ID_Cliente': 'nunique'})
```



Importancia de merge() en preparación de datos

El uso de merge() es fundamental en la consolidación de datos y en el preprocesamiento para modelos de machine learning, asegurando que toda la información relevante esté integrada en un solo DataFrame.

La capacidad de combinar datos de diferentes fuentes permite crear conjuntos de datos más completos y representativos, mejorando la calidad de los análisis y modelos predictivos.

alkemy

Caso de estudio: Análisis de ventas

Escenario

Una empresa de comercio electrónico necesita analizar el rendimiento de sus productos por categoría y región.

Datos disponibles

- Tabla de ventas (transacciones)
- Tabla de productos (información detallada)
- Tabla de clientes (datos demográficos)

Solución con Pandas

1. Combinar tablas usando merge()
2. Agrupar datos por categoría y región
3. Pivotar resultados para crear reportes
4. Visualizar tendencias y patrones

Implementación del caso de estudio

```
# 1. Combinar tablasdf_completo = pd.merge(ventas, productos,
on='ID_Producto')df_completo = pd.merge(df_completo, clientes, on='ID_Cliente')# 2.
Agrupar por categoría y regiónanalisis = df_completo.groupby(['Categoría',
'Región']).agg({ 'Monto': ['sum', 'mean'], 'ID_Venta': 'count'}).reset_index()# 3.
Pivotar para crear reporte por regiónreporte_regiones = analisis.pivot_table(
index='Categoría', columns='Región', values=('Monto', 'sum'))# 4. Visualizar
resultadosimport matplotlib.pyplot as pltreporte_regiones.plot(kind='bar',
figsize=(12, 6))plt.title('Ventas por categoría y región')plt.ylabel('Monto total de
ventas')plt.tight_layout()plt.show()
```

Optimización de operaciones con datos grandes

Filtrar antes de combinar

Reduce el tamaño de los DataFrames antes de realizar operaciones de merge o concatenación para mejorar el rendimiento.

Utilizar índices

Establece como índices las columnas que se utilizarán para combinar, lo que puede acelerar significativamente las operaciones de merge.

Controlar el uso de memoria

Utiliza dtypes apropiados para cada columna y considera el uso de chunks para procesar grandes volúmenes de datos por partes.

Buenas prácticas en agrupamiento y pivoteo

Documentación clara

Documenta las transformaciones realizadas para facilitar la reproducibilidad y comprensión del código.

Verificación de datos

Comprueba la integridad de los datos antes y después de las transformaciones para evitar errores.

Optimización de rendimiento

Considera el impacto en memoria y tiempo de ejecución, especialmente con conjuntos de datos grandes.

Manejo de valores nulos

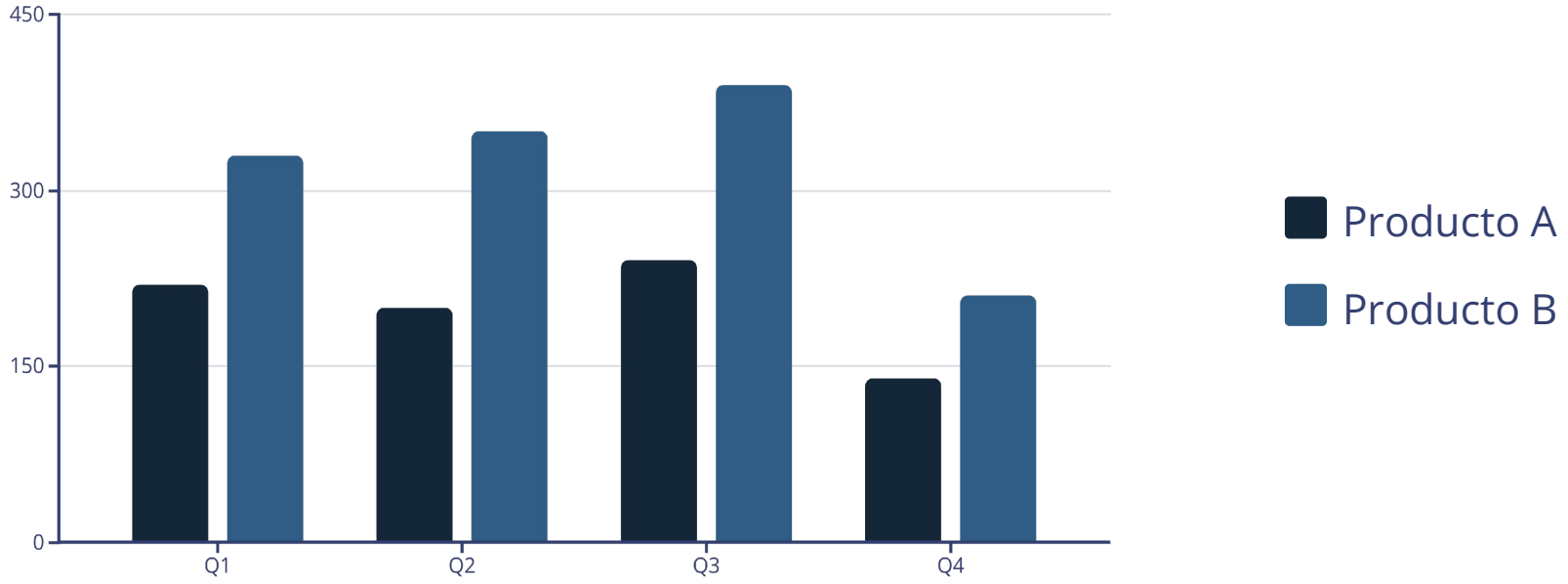
Define estrategias claras para el tratamiento de valores nulos en las operaciones de agrupamiento y pivoteo.

Ejercicio práctico: Análisis de ventas por trimestre

Supongamos que tenemos un DataFrame con datos de ventas mensuales por producto y queremos crear un informe trimestral.

```
# Datos de ejemplodata = { 'Fecha': pd.date_range(start='2023-01-01', periods=12, freq='M'),
'Producto': ['A', 'B', 'A', 'B', 'A', 'B', 'A', 'B', 'A', 'B', 'A', 'B'], 'Ventas': [100, 150,
120, 180, 90, 160, 110, 190, 130, 200, 140, 210]}df = pd.DataFrame(data)# Añadir columna de
trimestredf['Trimestre'] = df['Fecha'].dt.quarter# Agrupar por trimestre y productoventas_trim =
df.groupby(['Trimestre', 'Producto'])['Ventas'].sum().reset_index()# Pivotar para crear
informeinforme = ventas_trim.pivot(index='Trimestre', columns='Producto', values='Ventas')
```

Resultado del ejercicio práctico



El gráfico muestra las ventas trimestrales para cada producto. Podemos observar que el Producto B tiene consistentemente mayores ventas que el Producto A en todos los trimestres, con un pico en el tercer trimestre.

Desafíos comunes y soluciones

1

Valores duplicados en pivotado

Problema: Error al intentar pivotar datos con valores duplicados.

Solución: Utilizar pivot_table() en lugar de pivot() y especificar una función de agregación como 'sum' o 'mean'.

2

Memoria insuficiente

Problema: Operaciones que consumen demasiada memoria con grandes conjuntos de datos.

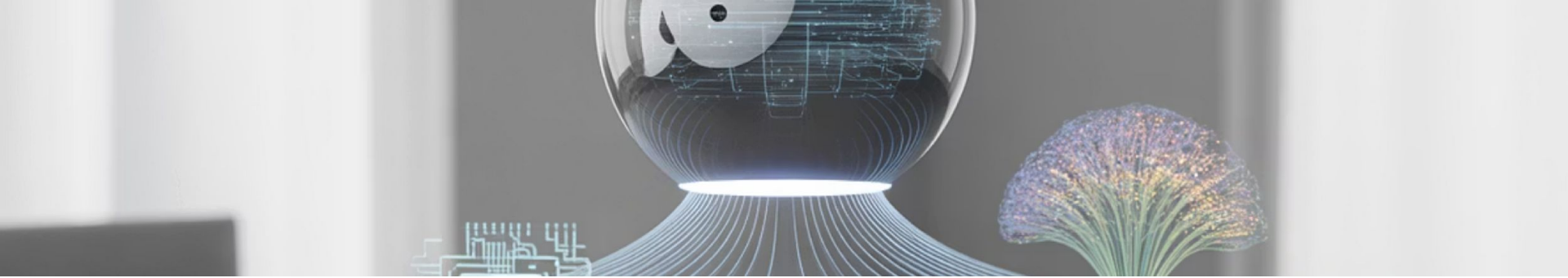
Solución: Procesar datos en chunks, optimizar tipos de datos (dtypes) o utilizar bibliotecas como Dask para procesamiento distribuido.

3

Claves no coincidentes en merge

Problema: Pérdida de datos importantes al realizar merge.

Solución: Verificar la integridad de las claves antes de la operación y utilizar el parámetro 'how' adecuado ('left', 'right', 'outer').



Herramientas complementarias

Dask

Biblioteca para computación paralela que extiende Pandas para trabajar con conjuntos de datos más grandes que la memoria disponible.

Vaex

Biblioteca para visualización y exploración de grandes conjuntos de datos tabulares (hasta ~100GB) en memoria.

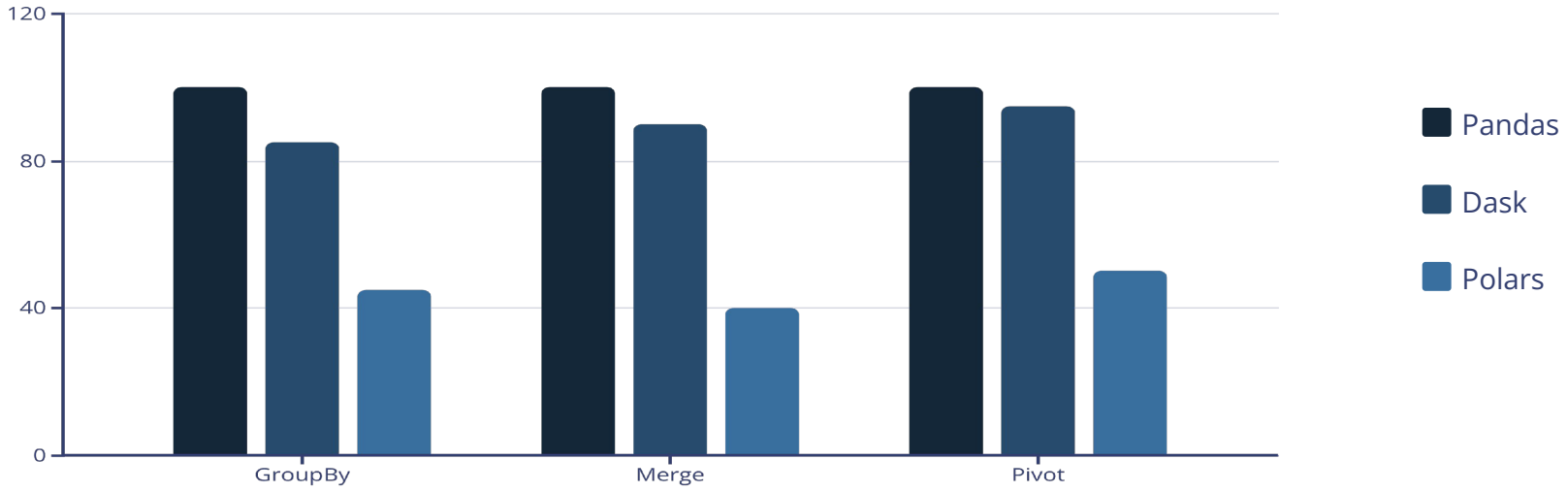
SQLAlchemy

Permite integrar operaciones SQL con Pandas, útil para trabajar con bases de datos relacionales.

Polars

Biblioteca de manipulación de datos rápida implementada en Rust, compatible con la API de Pandas pero con mejor rendimiento.

Comparación de rendimiento



El gráfico muestra el tiempo relativo de ejecución (menor es mejor) para diferentes operaciones en distintas bibliotecas, tomando Pandas como referencia (100). Polars muestra un rendimiento significativamente mejor en todas las operaciones.

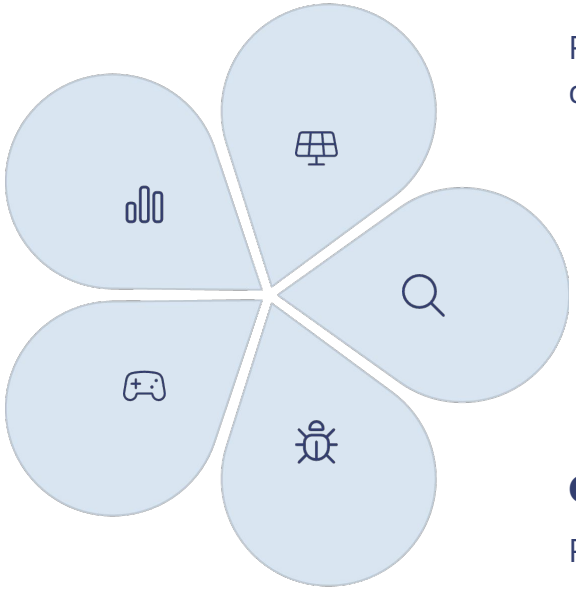
Aplicaciones en ciencia de datos

Análisis exploratorio

Agrupamiento y pivoteo para identificar patrones y tendencias en los datos.

Preparación para modelado

Estructuración de datos en el formato requerido por algoritmos de ML.



Limpieza de datos

Reorganización de datos para detectar y corregir inconsistencias.

Ingeniería de características

Creación de nuevas variables mediante agrupamiento y agregación.

Generación de reportes

Pivotado para crear tablas resumen y dashboards interactivos.

alkemy

Caso de uso: Análisis de datos financieros

Escenario

Un analista financiero necesita evaluar el rendimiento de diferentes inversiones a lo largo del tiempo y por sector.

Datos disponibles

- Transacciones diarias
- Información de instrumentos financieros
- Datos macroeconómicos

Solución con Pandas

1. Combinar fuentes de datos con merge().
2. Agrupar por sector y período
3. Calcular métricas de rendimiento
4. Pivotar para comparar sectores
5. Visualizar tendencias temporales

Implementación del análisis financiero

```
# 1. Combinar datosdf_completo = pd.merge(transacciones, instrumentos,
on='ID_Instrumento')df_completo = pd.merge(df_completo, macro_data, on='Fecha', how='left')# 2.
Calcular rendimiento diariodf_completo['Rendimiento'] =
df_completo.groupby('ID_Instrumento')['Precio'].pct_change()# 3. Agrupar por sector y
mesdf_completo['Mes'] = df_completo['Fecha'].dt.to_period('M')rendimiento_sector =
df_completo.groupby(['Sector', 'Mes'])['Rendimiento'].mean().reset_index()# 4. Pivotar para
comparar sectorescomparativa = rendimiento_sector.pivot(index='Mes', columns='Sector',
values='Rendimiento')# 5. Visualizar tendenciasimport matplotlib.pyplot as
pltcomparativa.plot(figsize=(12, 6))plt.title('Rendimiento mensual por
sector')plt.ylabel('Rendimiento promedio')plt.grid(True, alpha=0.3)plt.tight_layout()plt.show()
```

Tendencias y desarrollos futuros

Procesamiento distribuido

Mayor integración con herramientas de procesamiento distribuido como Spark y Dask para manejar conjuntos de datos a escala de petabytes.

Optimización de memoria

Nuevas estructuras de datos y algoritmos para reducir el consumo de memoria en operaciones de agrupamiento y pivoteo.

Integración con GPU

Aprovechamiento de GPUs para acelerar operaciones de manipulación de datos en Pandas y bibliotecas similares.

Interfaces declarativas

Desarrollo de APIs más intuitivas y declarativas para expresar transformaciones complejas de datos.

Recursos adicionales

Documentación oficial de Pandas

pandas.pydata.org - Guía completa con ejemplos detallados de todas las funcionalidades.

Python for Data Analysis

Libro de Wes McKinney (creador de Pandas) con explicaciones profundas sobre manipulación de datos.

Kaggle Learn

Cursos interactivos sobre Pandas con ejercicios prácticos y datasets reales.

Stack Overflow

Comunidad activa con soluciones a problemas específicos de manipulación de datos.

Ejercicios propuestos

Ejercicio 1: Análisis de ventas

Dado un dataset de ventas con columnas (Fecha, Producto, Categoría, Región, Monto):

1. Agrupa los datos por Región y Categoría
2. Calcula el total de ventas, promedio y conteo
3. Pivota los resultados para comparar categorías por región

Ejercicio 2: Análisis de rendimiento

Con un dataset de empleados (ID, Departamento, Proyecto, Horas, Fecha):

1. Combina con un dataset de proyectos (Proyecto, Prioridad, Cliente)
2. Agrupa por Departamento y Prioridad
3. Calcula horas totales y promedio por grupo
4. Crea un informe pivotado por Departamento y Prioridad

Preguntas frecuentes

¿Cuándo usar pivot() vs pivot_table()?

Use pivot() cuando esté seguro de que no hay duplicados en las combinaciones de índice/columna. Use pivot_table() cuando pueda haber duplicados y necesite especificar una función de agregación.

¿Cómo manejar valores nulos en operaciones de agrupamiento?

Puede usar dropna=False en groupby() para incluir grupos con valores nulos, o aplicar fillna() antes de agrupar para reemplazar los valores nulos con un valor predeterminado.

¿Qué hacer si merge() consume demasiada memoria?

Considere filtrar los DataFrames antes de combinarlos, usar dtypes optimizados, o implementar la operación en chunks procesando subconjuntos de datos por separado.

Más preguntas frecuentes

¿Cómo aplicar múltiples funciones a diferentes columnas en groupby()?

Use el método .agg() con un diccionario que especifique las funciones para cada columna: df.groupby('grupo').agg({'col1': ['sum', 'mean'], 'col2': 'count'})

¿Cómo realizar un self-join en Pandas?

Puede hacer merge de un DataFrame consigo mismo: pd.merge(df, df, left_on='col1', right_on='col2'). Esto es útil para encontrar relaciones dentro del mismo conjunto de datos.

¿Cómo optimizar el rendimiento de groupby() con muchos grupos?

Considere usar categoricals para columnas con muchos valores únicos, o implementar una estrategia de procesamiento por lotes si el número de grupos es extremadamente grande.

Resumen de técnicas clave

1. **Indexación jerárquica**
   Organización de datos en múltiples niveles para facilitar el acceso y análisis estructurado.

2. **Agrupamiento con groupby()**
   Segmentación de datos y aplicación de funciones agregadas para obtener estadísticas por grupo.

3. **Pivotado con pivot()**
   Transformación de datos de formato largo a ancho para facilitar la visualización y comparación.

4. **Despivotado con melt()**
   Conversión de datos de formato ancho a largo para normalización y análisis.

5. **Concatenación con concat()**
   Unión de DataFrames similares por filas o columnas sin necesidad de claves comunes.

6. **Combinación con merge()**
   Integración de datos de diferentes fuentes basada en claves comunes, similar a joins en SQL.

Mejores prácticas para el rendimiento

50%

Reducción de memoria

Optimizar tipos de datos puede reducir el uso de memoria hasta en un 50%.

10x

Aceleración con índices

Establecer índices adecuados puede acelerar operaciones de merge hasta 10 veces.

30%

Filtrado previo

Filtrar datos antes de agrupar o combinar puede mejorar el rendimiento en un 30%.

5x

Procesamiento en chunks

Procesar datos en lotes puede ser hasta 5 veces más eficiente en memoria para conjuntos grandes.

Flujo de trabajo recomendado

Exploración inicial

Examinar la estructura y contenido de los datos para comprender sus características.

Limpieza y preparación

Manejar valores nulos, duplicados y transformar tipos de datos según sea necesario.

Integración de fuentes

Combinar datos de diferentes orígenes mediante concat() o merge() según corresponda.

Transformación estructural

Aplicar técnicas de agrupamiento, pivotado o despivotado para reorganizar los datos.

Análisis y visualización

Utilizar los datos transformados para generar insights y visualizaciones informativas.

Herramientas complementarias de visualización

Matplotlib

Biblioteca fundamental para visualizaciones estáticas en Python, ideal para gráficos científicos y de análisis.

Plotly

Permite crear visualizaciones interactivas y dashboards web, ideal para exploración dinámica de datos.

Seaborn

Construida sobre Matplotlib, facilita la creación de visualizaciones estadísticas atractivas con menos código.

Bokeh

Enfocada en visualizaciones interactivas para navegadores web, con capacidades de zoom y exploración.

alkemy

Integración con ecosistema de ciencia de datos

Pandas

Manipulación y análisis de datos tabulares

SQLAlchemy

Integración con bases de datos

Matplotlib/Seaborn

Visualización de datos

NumPy

Computación numérica eficiente

Scikit-learn

Algoritmos de machine learning

Jupyter

Entorno interactivo de desarrollo

Cierre

El agrupamiento, pivotado y combinación de datos son técnicas fundamentales en el análisis y manipulación de datos estructurados. Estas herramientas permiten transformar conjuntos de datos en formatos que facilitan su exploración, interpretación y modelado, asegurando que la información esté organizada de manera óptima para la toma de decisiones.

El uso de la indexación jerárquica y groupby en Pandas facilita la agregación y segmentación de datos, proporcionando un marco flexible para analizar métricas en diferentes categorías. Por otro lado, el pivotado y despivotado de DataFrames permiten reorganizar la información en función de las necesidades del análisis, ya sea para generar reportes más claros o para preparar datos en estructuras normalizadas.

Live Coding

¿En qué consistirá la Demo?

Resumen automático de rendimiento por equipo

1. *Agrupar rendimiento de equipos.*
2. *Filtrar sólo los que superan cierto umbral.*
3. *Reordenar columnas y exportar el reporte.*

Tiempo: 25 minutos

#Momentode Preguntas...

¿ ?

¿Cómo se manejan valores nulos en agrupaciones?

¿ ?

¿Cómo aplicar filtros después de agrupar datos?

¿ ?

¿Cómo reordenar columnas de un DataFrame?

¿ ?

¿Qué pasa si se aplica .agg() sin renombrar?

Momento:

Time-out!

⏰5 -10 min.

alkemy

Ejercicio N° 1

Ranking de desempeño

Ranking de desempeño

Consigna: ✍️

1. Agrupar rendimiento por departamento.
2. Calcular promedio, mínimo y máximo de puntuación.
3. Filtrar resultados mayores a un umbral.
4. Ordenar de mayor a menor.
5. Preparar reporte final listo para exportar a Excel.

Tiempo 🕐**:** *40 minutoS*

Plataforma : Jupyter Notebook

¿Alguna consulta?

alkemy

Resumen

¿Qué logramos en esta clase?

- ✓ ***Limpiaste y manipulaste resultados de .groupby().***
- ✓ ***Aplicaste filtros condicionales tras agrupamiento.***
- ✓ ***Generaste reportes listos para visualización.***
- ✓ ***Automatizaste un flujo de resumen de datos.***

¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las ***actividades disponibles en la plataforma.*** *Estas propuestas son clave para afianzar lo trabajado y* ***forman parte obligatoria del recorrido de aprendizaje.***

💪 Análisis de caso ——— 💪 Selección Múltiple

💪 Comprensión lectora

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

alkemy

< ¡Muchas gracias! >

alkemy